Belt-Hook 參數結構 v2 vs v3 對照

> 本文件給已經依 v2 實作的後台 / 整合商，快速對照 v3 變了哪些東西、要注意什麼。完整 v3 規格請見 belt_hook_parameter_protocol.md。

一句話總結

Wire ABI（byte 偏移、欄位寬度、payload 長度）完全相容 v2，但 byte 0 與 byte 1 的「意義」完全換了：

- v2 的 byte 0 / byte 1 是 `system_cycle_time` / `advertising_duration`，用來告訴 hook「系統循環秒數」與「BLE 廣播時間」。
- v3 把這兩個設定 改成韌體寫死的內部行為，不再由後台下發；那兩個 byte 因此釋出，重新指派為 `alarm_window_s`（警報窗口秒數）與 `flags`（旗標位元組），支援 belt 與 hook 之間透過 BLE 連線即時交換狀態（例如「作業員進入 SAFE 區」）。

換句話說：v2 byte 0 / 1 與 v3 byte 0 / 1 在 wire 上是同樣的位置 + 同樣的寬度（都是 uint8），但代表的東西不一樣，不能視為「同一個欄位改名」。

hook_settings_struct（17 bytes）— byte by byte 對照

Byte	v2 名稱	v2 用途	v3 名稱	v3 用途
0	`system_cycle_time`	系統循環秒數（hook 週期任務的執行間隔）	`alarm_window_s`	警報窗口秒數（1–30）。值 0 是「flag-only write」哨兵 — hook 不更動 settings flash，只處理 `flags` 副作用
1	`advertising_duration`	BLE 廣播持續時間	`flags`	旗標位元組，bit `0x80` = `IN_SAFE_ZONE`（pin counter 0）；其餘 bit 保留必須填 0。transient：hook 寫入 flash 前固定歸 0，不持久化
2	`x.sensing_type`	同 v3	`x.sensing_type`	不變
3–4	`x.sensitivity_high`	同 v3	`x.sensitivity_high`	不變
5–6	`x.sensitivity_low`	同 v3	`x.sensitivity_low`	不變
7–11	`y.*`	同 v3	`y.*`	不變
12–16	`z.*`	同 v3	`z.*`	不變

> v3 端的 `system_cycle_time` / `advertising_duration` 去哪了？ 內化到韌體：hook 端的循環任務間隔與 BLE 廣播時間都由韌體常數決定（mag task = 500 ms、BLE 廣播 always-on 等），不再開放後台調整。如果未來需要重新放出來，會透過完全不同的欄位或新版協定處理，不會再用這兩個 byte。

外層結構名稱

名稱	v2	v3
內層 settings	`hook_settings_struct_v2` / `_v2_t`	`hook_settings_struct_v3` / `_v3_t`
BLE 17-byte 寫入	`hook_downlink_command_struct_v2`	`hook_downlink_command_struct_v3`

C 端 typedef name 跟著版次走，舊程式碼直接 `#include "data_types.h"` 編譯時會失敗，強制提示開發者去看 v3 的新 byte 0 / byte 1 語意，避免「能編過但行為錯」的隱性 bug。

belt_downlink_command_struct（74 bytes payload）— 不變

```
byte 0..23  belt header      24 bytes（reboot_and_ignore_reed、sleep、scan filter ...）
byte 24..48 hook[0]          25 bytes
byte 49..73 hook[1]          25 bytes
```

LoRaWAN fPort=30 收到的 payload 長度判斷規則（v2 / v3 共用）：

- 24 bytes → 只更新 belt 自己
- 74 bytes → 更新 belt + 兩支 hook
- 其他長度 → Belt 直接丟棄

v3 新增的兩個功能（v2 後台不會觸發）

1. Flag-only write（單純通知不改 settings）

```
hook_settings.alarm_window_s = 0x00    ← 哨兵
hook_settings.flags          = 0x80    ← 例：FLAG_IN_SAFE_ZONE
hook_settings.x / y / z      = 任意（hook 會忽略）
```

收到端	byte 0 = 0 的解讀	byte 1 = 0x80 的解讀
v2 hook	`system_cycle_time = 0`，視為設定值寫入；hook clamp 回 DEFAULT；XYZ 軸全 0 也會被存進 flash → 偵測會壞掉	`advertising_duration = 0x80`，當成 BLE 廣播設定值原樣存入；無實際行為
v3 hook	偵測到 byte 0 = 0 → 走 flag-only path，不寫入 flash；XYZ 全 0 被忽略	bit `0x80` = `IN_SAFE_ZONE` → 觸發 SAFE-zone counter pin

> 結論：v2 hook 收到 v3 後台送的 flag-only write 會把 XYZ 全寫成 0，等同把 hook 偵測弄壞 — 升級必須先燒 hook 再上 v3 後台（見下方升級檢查清單）。反向則 OK：v3 hook 收到 v2 後台原本送的真實 settings（byte 0 = 1–30、byte 1 = 廣播時間值），會 clamp byte 0 到合法範圍並把 byte 1 視為 flags（通常舊值 1 / 2 / 3 等不會湊到 `0x80`，所以不會誤觸 SAFE-zone）。

2. SAFE-zone counter pin（自動 / 手動）

Belt 韌體 v3 在偵測到 SAFE-role beacon 期間 自動 每 3秒對連線中的 hook 各送一次 flag-only write（`alarm_window_s = 0`、`flags = 0x80`），維持 hook 端 10秒的 grace window — counter / status 在 grace window 內被釘在 0，作業員在休息區不會觸發警報。

後台也可以用同樣的 fPort=30 payload 手動 觸發 reset counter（例如排程性的「重置作業日」）：

```
hook[i].id[6]            = 目標 hook MAC
hook[i].reserve          = 0x0000
hook[i].alarm_window_s   = 0x00
hook[i].flags            = 0x80
```

hook[i].x / y / z = 全 0
```

Belt 收到後會比對 ID、把 17-byte block 透過 BLE 轉送給對應 hook，hook 走 flag-only path、reset counter。

> v3 的核心設計變化：v2 把 belt → hook 的訊息侷限於「下發設定值」一條路；v3 額外新增「belt 與 hook 透過已連線的 BLE 通道，即時交換 transient 狀態」這條路。flag-only write 是這條路的第一個應用，未來可以在 `flags` 加 bit 擴充（例如 LED 模式、強制喚醒等）而不需要再拓出新欄位。

---

## 升級檢查清單

| 元件 | v2 → v3 需要改什麼 |
|---|---|
| Hook 韌體 | 必須 重新燒錄 v3 (理解 `alarm_window_s = 0` 哨兵 + `flags` bit 0x80；否則 v3 後台送 flag-only write 會打壞 hook 的 XYZ sensitivity) |
| Belt 韌體 | 重新燒錄 v3 (含 SAFE-zone 自動觸發 + 新 macro / struct 名稱) |
| 後台 / LoRa platform | 必要：把推到 hook 的 17-byte payload 中的 byte 0 / byte 1 對應欄位重新標示為 `alarm_window_s` / `flags`（值不需變動 — 之前的 `system_cycle_time` 數值通常落在 1–30，v3 hook 仍會視為 alarm window 接受） |
| 後台 / 想啟用 SAFE-zone 手動 reset | 在 hook block 模板上加「flag-only template」(`alarm_window_s=0`, `flags=0x80`) 給操作員選用 |
| 後台 payload generator (Python / Node) | 把 struct 定義裡的 byte 0 / byte 1 欄位名跟著改；wire bytes 不變所以不必動 generator 邏輯，但建議補註解標明這兩個 byte 在 v3 的新語意 |

升級順序：先 hook → 再 belt → 最後切換後台到 v3。先燒 belt 而還沒燒 hook 的話，belt 會送 flag-only write 給 v2 hook，造成 hook 的 XYZ sensitivity 被覆寫成 0。

---

## 相關文件

- [belt_hook_parameter_protocol.md](belt_hook_parameter_protocol.md) — v3 完整規格 + 範例 payload
- [belt_operation.md](belt_operation.md) — Belt 端使用者操作說明（含 SAFE / DANGER zone 行為）
- [`22_XDA003H/docs/hook_operation.md`](../hook/operation.md) — Hook 端使用者操作說明（含 SAFE-zone grace window 描述）
```